

DSA-in-C Prerequisites — Filtered Practice Set

Only the questions genuinely needed before starting Linked Lists, Stacks, Queues, Trees, and Graphs in C

Curated from ~400+ original questions down to ~80. Phases 1–6 are trimmed to warm-up only (you already have algorithmic thinking from JS/TS); Phases 7–10 are kept almost fully intact since they are the real gap for coding DSA in C.

Phase 1 — Conditional Thinking (SKIPPED)

Why: Pure if-else/boolean logic. You already have this from JS/TS — nothing C-specific here. No questions included.

Phase 2 — Looping & Patterns (Warm-Up Only)

Goal: Get comfortable with nested-loop index math in C syntax — this is reused constantly for matrix traversal, DP tables, and adjacency matrices.

Pattern Printing (Warm-Up)

1. Print a right-angled triangle of stars (n rows).
2. Print a pyramid of stars, centered (n rows).
3. Print a diamond pattern of stars.
4. Print a number triangle where row i contains 1 to i.

Phase 3 — Recursion

Goal: Base-case and recursive-decomposition thinking — required for trees, backtracking, and divide-and-conquer later.

Level 1 — Foundation

5. Print sum of first n natural numbers recursively.
6. Print factorial of a number recursively.
7. Find nth Fibonacci number recursively.

Level 2 — Number-Based Recursive Thinking

8. Count the number of digits in a number recursively.
9. Find GCD (HCF) of two numbers using Euclid's algorithm recursively.

Level 4 — String-Based Recursion

10. Reverse a string using recursion.
11. Check if a string is a palindrome using recursion.

Phase 4 — Basic Arrays

Goal: Traversal and aggregation fluency — these map directly onto real DSA array operations.

Level 1 — Fundamentals

12. Find the sum of all elements in an array.

13. Find the average of array elements.
14. Find the maximum element in an array.
15. Find the minimum element in an array.

Level 2 — Searching

16. Input an element x — check if it exists in the array (linear search).

Level 3 — Transformation

17. Reverse an array (without using built-in reverse).
18. Rotate an array by one position to the left.
19. Rotate an array by one position to the right.

Level 5 — Logical & Applied

20. Check if the array is sorted in ascending order.
21. Find the second largest element in an array.

Phase 5 — Strings (SKIPPED as a separate phase)

Why: Reverse-string and palindrome-check reappear in Phase 8 as pointer-based two-pointer problems — do them there. That version is the one that actually matters for DSA.

Phase 6 — Mixed Logical Challenges (SKIPPED)

Why: A remix of Phases 1–5 with no new DSA-relevant skill. If you did the phases above, this adds nothing.

Phase 7 — Functions (Modular Thinking)

Goal: Decompose logic into reusable functions — every DSA operation (insert, delete, search, traverse) is written as one.

Level 1 — Fundamentals

22. Write a function to add two numbers and return the result.
23. Write a function to swap two numbers (observe why call-by-value fails here).

Level 2 — Parameters & Return Logic

24. Write a function that takes an array and its size, and returns the sum.
25. Write a function that takes an array and returns the maximum element.
26. Write a function that takes an array and reverses it in place (void, modifies the caller's array).

Level 3 — Scope, Recursion & Applied Function Design

27. Demonstrate the difference between a local variable and a global variable using two functions.
28. Write a static local variable inside a function that counts how many times the function has been called.
29. Write a function `linearSearch(int arr[], int n, int key)` returning the index or -1.
30. Write a function `binarySearch(int arr[], int n, int key)` on a sorted array.
31. Write a function `bubbleSortStep(int arr[], int n)` that performs one pass of bubble sort — call it n times from main.

32. Design a small library of 3–4 functions (init, insert, display, sum) that operate on the same global array — a direct rehearsal for how a linked list or stack API will be structured in C.

Why: Every DSA operation in C — push(), pop(), insertNode(), search() — is a function. If function design and call-by-value/reference isn't second nature, DSA code becomes copy-pasted rather than understood.

Phase 8 — Pointers (Address & Memory Logic)

Goal: Understand memory addresses, dereferencing, and pointer arithmetic — the single most important C prerequisite for DSA. Do almost all of this phase.

Level 1 — Pointer Fundamentals

33. Declare an integer variable and print its address using &.
34. Declare a pointer to an int, assign it the address of a variable, and print the value via dereferencing.
35. Change the value of a variable indirectly through its pointer.
36. Demonstrate a NULL pointer and check for it before dereferencing (if (ptr != NULL)).
37. Take two integers and swap them using a function with pointer parameters (call by reference) — compare with the Phase 7 call-by-value attempt.
38. Declare a pointer to a pointer (int **pp) and access the original value through two levels of dereference.

Level 2 — Pointers with Arrays

39. Declare an array and a pointer to its first element; print elements using pointer arithmetic (*(ptr+i)) instead of arr[i].
40. Traverse an array using only a pointer and pointer increment (ptr++) in a loop.
41. Reverse an array in place using two pointers moving toward each other (this is the direct two-pointer technique used constantly in DSA).
42. Write a function that accepts an array as a pointer parameter (int *arr) and computes the average.

Level 3 — Pointers with Functions & Strings

43. Write a function that takes a char pointer and reverses the string in place using pointer swaps.
44. Write a function to check if a string is a palindrome using two pointers (start and end).

Level 4 — Applied Pointer Problems (Direct DSA Rehearsal, do all)

45. Simulate a single 'node' using two variables (int data, and an address-holder) and manually link two of them together — the exact idea behind a linked list node, done without struct first.
46. Write a function that takes a pointer to an integer and increments the value it points to, called from a loop to simulate a running counter across function calls.
47. Implement the two-pointer technique to find if an array has a pair that sums to a target value.
48. Implement a fast and slow pointer walk over an array (moving one pointer twice as fast) — this is the exact technique used later to detect cycles in linked lists.
49. Write a function that takes a pointer to an array and its length, and shifts all elements one position using pointer arithmetic only.
50. Simulate a simple 'swap by reference' utility function and use it inside a basic bubble sort implementation.
51. Write a function that returns a dynamically-sized array's pointer (conceptually — return the pointer from a function, understanding it must outlive the function's stack frame).
52. Use a pointer to traverse a 2D array (int arr[3][3]) row by row using pointer arithmetic instead of double indexing.
53. Write a function that takes a pointer to a pointer to swap which array a caller is 'looking at' (int **selector).

54. Trace through, on paper or in comments, what happens in memory when a pointer is passed through three nested function calls — write this as commented pseudocode inside a working C program.

Why: A linked list node is just a struct with a pointer to the next node. If pointer arithmetic and call-by-reference aren't fluent, every single linked-list/tree/graph operation in DSA will feel like magic instead of logic.

Phase 9 — Structures & User-Defined Data Types

Goal: Learn to group related data into a single unit — this is what a 'Node' actually is in C.

Level 1 — Struct Fundamentals

55. Define a struct Student with name, roll number, and marks; declare a variable and print its members.
56. Use typedef to simplify a struct declaration (typedef struct {...} Point;).
57. Create an array of 5 struct Student and fill it using a loop; print all records.

Level 2 — Structures with Pointers (Direct Node Rehearsal, do all four)

58. Declare a pointer to a struct Student and access its members using the arrow operator (->).
59. Define a struct Node with an int data field and a struct Node* next field — this IS a linked list node; do not link anything yet, just declare it correctly.
60. Create two struct Node variables manually and set the first node's next pointer to the address of the second — a two-node manual chain, traversed and printed with a loop.
61. Write a function createNode(int value) that allocates a struct Node on the stack (not yet with malloc) and returns it by value.

Why: struct Node { int data; struct Node* next; }; is the literal definition of a linked list node. Every tree, graph, stack, and queue implementation in C starts with a struct exactly like this.

Phase 10 — Dynamic Memory Allocation

Goal: Learn to create and destroy memory at runtime — required for every non-fixed-size DSA structure.

Level 1 — Fundamentals

62. Dynamically allocate a single integer using malloc, assign it a value, print it, then free it.
63. Dynamically allocate an array of n integers (size taken from user input) using malloc.
64. Demonstrate a dangling pointer: free a pointer, then explain (in comments, not by dereferencing) why using it afterward is undefined behavior.
65. Dynamically allocate memory for a struct Point using malloc and access its members using ->.

Level 2 — Dynamic Arrays & Resizing

66. Build a dynamic array that starts at capacity 2 and doubles its capacity using realloc whenever it's full — a minimal version of what a dynamic array / ArrayList does internally.

Level 3 — Applied: Building Your First Real Node (do all five — this is the finish line)

67. Combine Phase 9's struct Node with malloc: write createNode(int value) that uses malloc to allocate a new node on the heap, sets data and next = NULL, and returns the pointer — this is the actual function you will reuse in every linked-list program from here on.
68. Create two nodes using createNode() and manually link them (first->next = second); traverse and print the 2-node chain, then free both nodes.

69. Write a function to create n nodes in a loop, linking each to the previous one, forming a simple chain (a linked list, informally, without naming it yet).
70. Traverse the chain built above and free every node one at a time without losing the reference to the next node (the correct free-while-traversing pattern used to avoid memory leaks in real linked-list deletion).
71. Write a function `countNodes(struct Node *head)` that walks the chain and returns its length — your first real linked-list-style function.

Why: `malloc + struct + pointer` is the exact combination behind every 'insert a node' operation in linked lists, trees, and graphs. Phase 10, Level 3 is quite literally your first linked list, built with full understanding instead of copied syntax.

Readiness Checklist Before Starting DSA in C

72. Comfortable writing and calling functions with parameters and return values.
73. Comfortable using `&` and `*` without hesitation, and explaining call-by-value vs call-by-reference.
74. Comfortable declaring a struct with a self-referential pointer (`struct Node* next`) and explaining what it means.
75. Comfortable allocating memory with `malloc`, checking for `NULL`, and freeing it — without leaks or dangling pointers.
76. Have manually built and freed a small linked chain of nodes (Phase 10, Level 3) at least once.

Once all five are true, you are genuinely ready — not just theoretically, but practically — to start Linked Lists, Stacks, Queues, Trees, and Graphs in C.